

الأسبوع الأول – الكود النظيف

تقرير شامل حول مفاهيم البرمجة النظيفة وأساسيات كتابة كود احترافي قابل للصيانة والتطوير

CLEAN CODE

WEEK 1

الكود النظيف – Clean Code

ما هو الكود النظيف؟

الكود النظيف هو أسلوب كتابة برمجيات يهدف لجعل الكود سهل القراءة، الفهم، والتعديل. يعتمد على وضوح الأسماء، تنظيم بسيط، وتركيب منطقي يشرح نفسه بنفسه.

لماذا وُجد؟

الكود النظيف جاء عشان يحل مشكلة حقيقية: الكود الفوضوي صعب التعديل، يضيع وقتك، ويكثر الأخطاء. ببساطة، وُجد عشان تخفف الصداع اللي يجيك لما ترجع تشتغل على كود قديم.

مثال: الكود غير النظيف

```
x = 5  
y = 10  
z = x + y  
print(z)
```

مثال: الكود النظيف

```
first_number = 5  
second_number = 10  
sum_result = first_number +  
second_number  
print(sum_result)
```

✓ الفرق: المقصود واضح دون حاجة لتعليقات.

هيكلية المشروع – Project Structure

هيكلية المشروع هي الطريقة التي تُنظم بها المجلدات والملفات داخل المشروع بناءً على وظائفها، مثل فصل واجهة المستخدم عن منطق العمل عن قاعدة البيانات. وُجدت هذه الهيكلية لحل مشكلة الفوضى في الملفات — فبدونها، يصبح العثور على ملف معين أو إضافة ميزة جديدة كالبحث عن إبرة في كومة قش.

مثال: مشروع ويب منظم

```
my_project/
├── src/
│   ├── controllers/ # منطق استقبال الطلبات
│   ├── models/      # التعامل مع قاعدة البيانات
│   ├── views/       # واجهة المستخدم
│   └── utils/        # دوال مساعدة
├── tests/            # اختبارات المشروع
└── README.md
```



بدون هيكلية، ستجد كل الملفات في مجلد واحد، مما يعيق التطوير ويجعل المشروع صعب الصيانة.

قابلية التوسع 🚀

إضافة ميزات جديدة دون فوضى

سهولة التنقل 🔍

العثور على أي ملف بسرعة

فصل المسؤوليات 📁

كل مجلد مسؤول عن وظيفة واحدة

التسميات – Namings

❌ تسمية سيئة

ما هي التسمية الجيدة؟

التسمية الجيدة هي اختيار أسماء للمتغيرات، الدوال، والملفات بحيث تعكس دورها الحقيقي بسهولة، دون الحاجة لشرح إضافي.

```
def d(a, b):  
    return a * b / 2
```

لماذا وُجدت؟

تحل مشكلة الغموض — فالأسماء العامة مثل `data` أو `temp` أو `func1` لا تخبر القارئ بشيء عن محتواها أو وظيفتها، مما يضطره لقراءة الكود كاملاً لفهمها.

✅ تسمية جيدة

```
def calculate_triangle_area(base, height):  
    return base * height / 2
```

❗ الأسماء الواضحة تجعل الكود أشبه بوثيقة تقرأ من الأعلى للأسفل.

بدون تعليقات

الكود الجيد لا يحتاج لشرح إضافي

أسماء المعاملات

`a` و `base` أوضح بكثير من `height` و `b`

وضوح المعنى

اسم الدالة يصف بالضبط ما تفعله

مبدأ DRY – Don't Repeat Yourself

مبدأ DRY يعني عدم تكرار نفس المنطق أو نفس المعلومات في أكثر من مكان في المشروع. يتم استخراج الكود المكرر إلى دالة أو وحدة واحدة يُعاد استخدامها. وُجد هذا المبدأ لحل مشكلة تضارب التحديثات: لو كان لديك نفس الحساب أو الشرط في 10 أماكن، وعليك تعديله لاحقًا، فستنسى أحدها حتمًا، مما يسبب أخطاءً يصعب اكتشافها.

❌ مخالف لـ DRY

```
# مكان أول
price_with_tax = price * 1.15

# مكان ثاني بعد 500 سطر
final_price = original * 1.15
```

✅ متوافق مع DRY

```
def add_tax(amount):
    return amount * 1.15

price_with_tax = add_tax(price)
final_price = add_tax(original)
```



أي تعديل لضريبة 15% يتم في مكان واحد فقط — وهذا هو جوهر مبدأ DRY.

خلاصة – المفاهيم الأربعة الأساسية

هذه المفاهيم الأربعة تشكل أساس كتابة كود مهني احترافي:



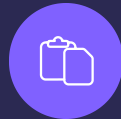
الهيكلية

تمنع الفوضى وتنظم الملفات بذكاء



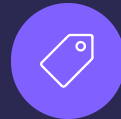
الكود النظيف

يقرأ كجمل إنجليزية واضحة وسلسلة



مبدأ DRY

يمنع ازدواجية العمل والأخطاء



التسميات

تشرح نفسها بنفسها دون تعليقات

الكود الجيد ليس مجرد كود يعمل — بل هو كود يُفهم ويُحافظ عليه ويُطوّر بسهولة.